
Desarrollo Web en Entorno Servidor - Segunda convocatoria

Desarrollo Web en Entorno Servidor - Segunda convocatoria

Desarrollo Web en Entorno Servidor – Segunda convocatoria

Ciclo: DAW (CFGS) · Módulo: DWES · Duración: 4 horas · Curso: 2025/2026

ANTES DE EMPEZAR

Qué vas a construir

HabitatStudio es una plataforma Laravel de alquiler de habitaciones. Ya tiene reservas (`bookings`), casas (`casas`) y usuarios (`users`). Tu trabajo es convertir los **testimonios estáticos** de `home.blade.php` en una funcionalidad real con base de datos, CRUD web y API REST.

Pasos previos obligatorios

- ☐ Hacer **fork** del repositorio **habitatstudio** y clonarlo.
- ☐ Crear la rama **segundaConvocatoria**.
- ☐ Configurar `.env` (parte de la nota – ver nota RA5e).
- ☐ Ejecutar `php artisan migrate:fresh --seed` para verificar que el entorno arranca.
- ☐ Al terminar: **Pull Request** de la rama **segundaConvocatoria**.

Reglas del examen

	Copilot deshabilitado (Disable Completions)
	No se puede usar IA para resolver los ejercicios
	Un ejercicio no puntúa si los anteriores no funcionan
	Esquemas de BD disponibles en el repositorio: inicial · final

Cómo están organizados los ejercicios

Hay tres bloques. Cada bloque tiene su propia nota, independiente de los demás.

Bloque	RA evaluado	Nota
A	RA5 – Blade / separación lógica-presentación	Independiente
B	RA6 – Acceso a base de datos	Independiente
C	RA7 – API REST	Independiente

Orden recomendado: A → B → C. Cada bloque se apoya en el anterior.

BLOQUE A – BLADE Y SEPARACIÓN LÓGICA/PRESENTACIÓN (RA5)

Objetivo del bloque: crear las migraciones, modelos y vistas Blade que dan soporte real a los testimonios.

Tiempo orientativo: 90 minutos

A1 – Clave ajena del propietario en `casas` · 1 pto.

Qué hay que hacer:

- ☐ Crear una migración `propietario_to_fk_in_casas_table` que:
 - Elimine la columna `nombre_propietario` de la tabla `casas`.
 - Añada una clave ajena `propietario_id` que referencie a `users`.
- ☐ En el modelo `Casa`: añadir la relación `propietario()` (`belongsTo User`).
- ☐ En el modelo `User`: añadir la relación inversa `casas()` (`hasMany Casa`).

 No toques la migración original de `casas`. Crea una migración nueva.

Cómo saber que está bien: `php artisan migrate:fresh` no da errores y `Casa::find(1)->propietario` devuelve un `User`.

A2 – Campos de perfil en `users` · 1 pto.

Qué hay que hacer:

- ☐ Crear una migración `add_profile_fields_to_users_table` que añada a `users`:
 - `avatar_url` – `varchar`, nullable.
 - `descripcion_perfil` – `varchar`, nullable.

 No toques la migración original de `users`. Crea una migración nueva.

Cómo saber que está bien: `User::find(1)->avatar_url` no da error (devuelve `null` o una URL).

A3 – Migración y modelo `Testimonio` · 1 pto.

Qué hay que hacer:

- ☐ Crear la migración de la tabla `testimonios` con estas columnas:

Columna	Tipo	Notas
<code>user_id</code>	FK → <code>users</code>	Obligatorio
<code>casa_id</code>	FK → <code>casas</code>	Nullable (null = testimonio sobre la plataforma)
<code>contenido</code>	<code>text</code>	Obligatorio
<code>valoracion</code>	<code>decimal(3,1)</code>	Entre 0.0 y 5.0
<code>fecha_aprobacion</code>	<code>timestamp</code>	Nullable (null = pendiente de aprobación)

La tabla incluye también `id` y `timestamps`.

- ☐ Crear el modelo `Testimonio` con:
 - `usuario()` – `belongsTo User`
 - `casa()` – `belongsTo Casa`
- ☐ En `User`: añadir `testimonios()` – `hasMany Testimonio`.
- ☐ En `Casa`: añadir `testimonios()` – `hasMany Testimonio`.

Cómo saber que está bien: `Testimonio::create([...])` funciona sin errores y `$testimonio->usuario->name` devuelve el nombre del usuario.

A4 – Seeder · 1 pto.

Qué hay que hacer:

- ☐ Crear `TestimonioSeeder` que inserte **al menos 5 testimonios** cumpliendo:
 - Mínimo 2 con `fecha_aprobacion` rellena.
 - Mínimo 1 con `casa_id` a `NULL`.
 - Mínimo 1 con valoración decimal (p. ej. `4.5`).
 - Todos los `user_id` deben ser de usuarios que tengan al menos una `booking`.
- ☐ Registrar `TestimonioSeeder` en `DatabaseSeeder` para que se ejecute con `migrate:fresh --seed`.

Cómo saber que está bien: `php artisan migrate:fresh --seed` termina sin errores y `Testimonio::count()` devuelve 5 o más.

A5 – Vista dinámica de testimonios · 2 ptos.

Qué hay que hacer:

Parte a) – Controlador - [] Modificar el método del controlador que carga `home.blade.php` para que recupere solo los testimonios con `fecha_aprobacion` no nula y los pase a la vista.

Parte b) – Partial - [] Modificar `resources/views/partials/testimonios.blade.php` para que recorra la colección de testimonios y muestre para cada uno: - Avatar (`avatar_url`). Si es `null`, mostrar un avatar por defecto. - Nombre del usuario y `descripcion_perfil`. - Contenido del testimonio.

Cómo saber que está bien: la portada muestra los testimonios del seeder con `fecha_aprobacion` rellena y no muestra los que están pendientes.

A6 – (No obligatorio) Estrellas de valoración · 2 ptos.

Qué hay que hacer:

- ☐ Añadir al modelo `Testimonio` un método `estrellas()` que devuelva un array de 5 elementos con los valores `'full'`, `'half'` o `'empty'` según la valoración.
 - Estrella entera → `fa-solid fa-star`
 - Media estrella (decimal ≥ 0.5) → `fa-solid fa-star-half-stroke`
 - Estrella vacía → `fa-regular fa-star`
- ☐ Usar ese método en el partial para renderizar los iconos.

 La lógica debe estar en el **modelo**, no en la vista Blade.

A7 – (No obligatorio) Testimonios en la vista de una casa · 2 ptos.

Qué hay que hacer:

- ☐ En la vista `show` de una casa, mostrar los testimonios de esa casa que tengan `fecha_aprobacion` no nula.

BLOQUE B – ACCESO A BASE DE DATOS (RA6)

Objetivo del bloque: CRUD completo de testimonios con reglas de negocio.

Tiempo orientativo: 90 minutos

 **Requisito previo:** la tabla `testimonios` y el modelo `Testimonio` del Bloque A deben estar operativos.

B1 – Controlador y rutas · 1 pto.

Qué hay que hacer:

- ☐ Crear `TestimonioController` con todas las rutas resource bajo el prefijo `/testimonios`.

Cómo saber que está bien: `php artisan route:list | grep testimonios` muestra las rutas index, create, store, edit, update y destroy.

B2 – Listado y creación · 1 pto.

Qué hay que hacer:

`index` - [] Mostrar todos los testimonios del usuario autenticado (aprobados y pendientes), ordenados por fecha de creación descendente. - [] Incluir en cada tarjeta el estado: `Aprobado` o `Pendiente`.

`create` y `store` - [] El formulario (ya existe en `resources/views/testimonios/create.blade.php`, revisa qué le falta) debe incluir: - Desplegable con las casas disponibles + opción «Comentario general sobre la plataforma» (valor vacío → `NULL` en `casa_id`). - Campo de texto para el contenido. - Campo numérico para la valoración (0–5, paso 0.5). - [] El método `store` debe: - Asignar `user_id` del usuario autenticado (ignorar cualquier `user_id` que venga en la petición). - Poner `fecha_aprobacion` a `NULL`. - Validar: `contenido` no vacío · `valoracion` entre 0 y 5 · `casa_id` existe en `casas` (si se envía).

B3 – Edición y eliminación · 2 ptos.

Qué hay que hacer:

`edit` y `update` - [] Permitir editar un testimonio propio **solo si no está aprobado** (`fecha_aprobacion` es `NULL`). - [] Si ya está aprobado: redirigir al `index` con mensaje de error.

`destroy` - [] Eliminar el testimonio que coincida con el `id` y pertenezca al usuario autenticado.

⚠ Si el usuario intenta actuar sobre un testimonio que no es suyo: responder `403`.

B4 – Restricción a inquilinos · 2 ptos.

Qué hay que hacer:

☐ Solo los usuarios con al menos una `booking` como `inquilino_id` pueden crear, editar o eliminar testimonios.

☐ Si un usuario autenticado no cumple esa condición: responder `403`.

💡 Puedes añadir un método `esInquilino()` al modelo `User` que compruebe si existe alguna `booking` con su `id`.

B5 – (No obligatorio) Aprobación por el propietario · 2 ptos.

Qué hay que hacer:

- ☐ Implementar la ruta `PUT /testimonios/{testimonio}/approve`.
- ☐ Solo puede ejecutarla el **propietario de la casa** a la que hace referencia el testimonio.
- ☐ Si funciona correctamente: actualizar `fecha_aprobacion` con `now()` y redirigir con mensaje.
- ☐ Si el testimonio tiene `casa_id` a `NULL`: responder `422`.
- ☐ Implementar la autorización con una `Policy` sobre `Testimonio`, método `approve`.

B6 – (No obligatorio) Revisión por el administrador · 2 ptos.

Qué hay que hacer:

- ☐ Implementar la ruta `GET /testimonios/{testimonio}/revisar`.
- ☐ Solo puede acceder el **administrador** (el usuario cuyas credenciales están en `.env`).
- ☐ Devuelve una vista con el testimonio y un formulario para **aprobar** o **rechazar**:
 - Aprobar: pone `fecha_aprobacion` a `now()` y redirige con mensaje.
 - Rechazar: elimina el testimonio y redirige con mensaje.
- ☐ Implementar la autorización con una `Policy` sobre `Testimonio`, con un método `before` que devuelva `true` si el usuario es administrador.

BLOQUE C – API REST (RA7)

Objetivo del bloque: exponer los testimonios como API REST con Sanctum.

Tiempo orientativo: 60 minutos

 **Requisito previo:** el modelo `Testimonio` y la lógica del Bloque B deben estar operativos.

C1 – Listado público · 1 pto.

Qué hay que hacer:

- ☐ Crear `TestimonioResource` que exponga: `id`, `contenido`, `valoracion`, `fecha_aprobacion`, `name` y `avatar_url` del usuario, y `nombre_casa` si existe.
- ☐ Implementar el endpoint **público** (sin autenticación):

```
GET /api/v1/testimonios
```

Devuelve todos los testimonios con `fecha_aprobacion` no nula, ordenados por `fecha_aprobacion` descendente.

C2 – Testimonios de una casa · 1 pto.

Qué hay que hacer:

- ☐ Implementar el endpoint **público**:

```
GET /api/v1/casas/{casa}/testimonios
```

Devuelve los testimonios aprobados de esa casa. Si la casa no existe: 404 .

C3 – Creación autenticada · 2 ptos.

Qué hay que hacer:

- ☐ Implementar el endpoint **autenticado** (Sanctum):

```
POST /api/v1/testimonios
```

Con las mismas reglas que B2 y B4: - [] `user_id` del usuario autenticado (no del cuerpo de la petición). - [] `fecha_aprobacion` a `NULL`. - [] Solo inquilinos; si no: 403 . - [] Respuesta correcta: `TestimonioResource` con código 201 .

C4 – CRUD completo con Policy · 3 ptos.

Qué hay que hacer:

- ☐ Implementar los endpoints **autenticados** (Sanctum):

```
PUT /api/v1/testimonios/{testimonio}
DELETE /api/v1/testimonios/{testimonio}
PUT /api/v1/testimonios/{testimonio}/approve
```

Con estas reglas (misma `Policy` que en B3/B5, adaptada a respuestas JSON):

Acción	Quién puede	Respuesta correcta	Error
<code>update</code>	Autor + no aprobado	<code>TestimonioResource</code> actualizado	403
<code>destroy</code>	Autor	204 No Content	403
<code>approve</code>	Propietario de la casa	<code>TestimonioResource</code> actualizado	403 / 422 si <code>casa_id</code> es <code>NULL</code>

⚠ Los errores de autorización deben devolver `403` en JSON, no una redirección.

C5 – (No obligatorio) Documentación OpenAPI/Swagger · 3 ptos.

Qué hay que hacer:

- ☐ Documentar los endpoints de C1 a C4 con OpenAPI/Swagger incluyendo: descripción, parámetros, esquemas de respuesta y posibles errores.

RESUMEN DE PUNTUACIÓN

Bloque A – RA5

Ej.	Descripción	Ptos.
A1	Clave ajena propietario en <code>casas</code>	1
A2	Campos de perfil en <code>users</code>	1
A3	Migración y modelo <code>Testimonio</code>	1
A4	<code>TestimonioSeeder</code>	1
A5	Vista dinámica con partial	2
A6 (no oblig.)	Estrellas de valoración	2
A7 (no oblig.)	Testimonios en vista de casa	2
Total RA5		10

Bloque B – RA6

Ej.	Descripción	Ptos.
B1	Controlador y rutas	1
B2	Listado y creación	1
B3	Edición y eliminación	2
B4	Restricción a inquilinos	2
B5 (no oblig.)	Aprobación por propietario	2
B6 (no oblig.)	Revisión por administrador	2
Total RA6		10

Bloque C – RA7

Ej.	Descripción	Ptos.
C1	Resource y listado público	1
C2	Testimonios de una casa	1
C3	Creación autenticada	2
C4	CRUD completo con Policy	3
C5 (no oblig.)	OpenAPI/Swagger	3
Total RA7		10